System Audit

Fin Cash Development Progress

Elsewedy School | Team code team13
Supervisor: Mostafa Radian | Report date: 24 October 2026

Team member	Code
Marwan Hazem	524069
Yomna Reda	524042
Youssef Mohamed	524005
Sama Adel	524033
Mohamed Walid	524001

Purpose and Current Scope

This document records our earlier project work, the user needs identified during project evaluation, and the two features selected to address them: Quick Search & Filter and Monthly Financial Summary. Our current development scope is limited to these two features.

Completed Preparation Activities

We renamed and described the project as Fin-Cash, prepared test cases for the main financial functions, and ran the project to check frontend and API integration. We then completed user-research and requirements activities and organized the work on Trello.

Completed activity	Recorded completion
Project name and description	15 September 2026
Test-case preparation	17 September 2026
Project startup and integration check	19 September 2026
Empathy Map	21 September 2026
Affinity Map	23 September 2026
Software Requirements Specification	26 September 2026
Capstone Development Timeline	28 September 2026
Trello task organization	30 September 2026

These completion dates follow our earlier project timeline. Portfolio is recorded separately as In Progress and is not counted as completed.

Project Evaluation and Identified Needs

What We Worked on After Preparation

After the earlier preparation work, we reviewed the existing project and evaluated how its financial reports, transaction records and dashboard could support the next development stage. We examined the frontend/API interaction and started planning the design and architecture for the two selected features.

The earlier team timeline records project testing and test-case preparation. The findings below describe the current feature gaps identified from the code and the agreed requirements; they are not new passing runtime-test results.

Need 1 Finding the Right Transaction

Users need a quick way to locate a financial transaction and distinguish revenue from expenses. Searching manually through reports and records makes it harder to isolate the entries relevant to a particular task.

The existing frontend already provides quick search within the report grid and filters such as year, category and status. However, report search does not fully satisfy the requested transaction-level search, and the current transaction record does not define an explicit Revenue/Expense type.

Our selected solution is Quick Search & Filter: a search bar and a transaction-type selector that work together, allowing users to view matching transactions rather than inspect the full list manually.

Need 2 Understanding a Selected Month

Users need one clear view of monthly revenue, expenses and their difference. A list of financial records or a general chart does not directly provide the three requested values for a selected month.

The current dashboard groups report totals by report creation month and treats the totals as expenses. A report can contain transactions from different months, so that calculation does not provide the requested summary based on individual transaction dates.

Our selected solution is Monthly Financial Summary: a month/year selector and three dashboard cards calculated from eligible transactions within the selected month.

Design Direction

We will build on the existing React interface and ASP.NET Core API. Search and summary calculations should use the same definition of Revenue and Expense. Existing API functionality will be reused where it meets the requirements; missing type and query support will be added as needed.

Code references: Frontend/components/financial-reports-view.tsx:425–428; Frontend/components/dashboard-view.tsx:135–158; Backend/Models/TransactionRecord.cs:6–24.

Feature 1 Quick Search and Filter

Problem and Intended Improvement

The feature addresses the difficulty of finding a specific transaction and separating income entries from expense entries. It extends the current report-level search to the transaction information required by the user.

User Experience

The user enters a transaction name or identifying description in a search bar. A nearby selector provides All, Revenue and Expense. Selecting Expense displays expense transactions only; entering a search term at the same time further limits those results.

For example, the user selects Expense and searches for office supplies. The list displays expense entries matching that description. Clearing the search and choosing All restores the default list. If nothing matches, the interface displays a clear no-results message.

Backend and Frontend Design

The current record uses Description rather than a separate Name field. We will agree on that searchable field before finalizing the request/response contract. A report title can remain a separate search target if required.

An explicit transaction type will distinguish Revenue from Expense in the record model, DTOs and database. Existing entries must be classified deliberately; category alone will not be assumed to define transaction type.

The API will apply the agreed search and type criteria together and return matching accessible records. React will provide the search bar, selector and loading/empty states. We will reuse existing query support where possible; the current code still needs transaction-type support.

Expected Benefits

Users can reach relevant entries with fewer manual steps, distinguish revenues from expenses clearly, and narrow the displayed data without reviewing unrelated transactions.

Acceptance Criteria

- Searching returns transactions matching the agreed identifying field.
- Revenue and Expense each return only the selected type; All removes the type restriction.
- Search and type filters work together, and resetting the controls restores the default results.
- No-match results are displayed clearly; users see only transactions they are permitted to access.

Current Progress

We have reviewed the existing search behavior and identified the missing transaction-type support. Design and architecture planning have started. Implementation and runtime acceptance testing are the next steps; the feature is not recorded as completed.

Feature 2 Monthly Financial Summary

Problem and Intended Improvement

The feature addresses the lack of a direct monthly view of total revenue, total expenses and the resulting balance. Users should obtain these figures without calculating them manually from individual entries.

User Experience

The dashboard will provide a month/year selector and three cards: Total Revenues, Total Expenses and Net Profit, using the requested interface labels. Selecting October 2026 will show the totals for transactions dated within that month.

The business definition of Net Profit must be confirmed: revenue minus recorded expenses is the displayed calculation, but its accounting meaning depends on the data included in Fin-Cash.

Calculation and Architecture

Both features will share the same Revenue/Expense classification. The API will calculate the monthly totals from TransactionDate, including the first day of the selected month and excluding the first day of the next month. Report status eligibility and currency rules must be agreed before acceptance.

Total Revenues is the sum of eligible Revenue entries; Total Expenses is the sum of eligible Expense entries; the third card shows Total Revenues minus Total Expenses. React will request the selected month and update the three cards. A month with no eligible transactions will display zero values.

For example, if October contains revenue of EGP 10,000 and expenses of EGP 6,000, the cards display EGP 10,000, EGP 6,000 and EGP 4,000. These are illustrative values, not project test results.

Acceptance Criteria

- Changing the month/year updates all three cards consistently.
- Totals use transaction dates, even when the parent report was created in another month.
- Revenue, expense and difference calculations match known test data; empty months show zero.
- The selected report-status and access rules apply consistently to the summary.

Team Progress and Next Steps

Completed: project preparation, earlier testing activities, user research, requirements, timeline and task organization. Reviewed: existing search and dashboard behavior. Started: design and architecture for the two selected features. Next: agree the shared transaction type, implement the API/UI changes, and test search, filtering and monthly calculations.

The two features are our selected solutions to the identified user needs. Their effectiveness will be confirmed after implementation and acceptance testing; they are not claimed as delivered in this progress report.